
--
Spring Cloud | Microservices Core Notes

--

>> Monolithic Architecture

-- In monolithic architecture, the entire application exists as a single unit. All modules (User, Product, Order, Payment, Auth) are part of the same project, use the same database, and are deployed together in one deployment.

==========

>> Structure Example

Ecommerce-App
├ User Module
├ Product Module
├ Order Module
├ Payment Module
└ Auth Module
Single DB + Single Deployment

Example : Amazon and Flipkart were monolithic

==========

>> Advantages

-- Simple to develop (single codebase)
-- Easy debugging
-- Easy testing
-- Transaction management easy

==========

>> Disadvantages

-- Large codebase → hard to maintain
-- Scaling only full app (costly)
-- Small change → full redeploy
-- Tech stack lock-in

--
Microservices Architecture

--

The application is divided into small, independent services.

Each service has :

>> A separate codebase
>> A separate database
>> Independent deployment

==========

>> Structure Example

User Service → user-db
Product Service → product-db

```
Order Service    →   order-db
Payment Service  →   payment-db
API Gateway      →   Entry point

===========

>>  Advantages

--  Independent deployment
--  Independent scaling
--  Fault isolation
--  Tech flexibility

===========

>>  Challenges

--  Complex architecture
--  Network latency
--  Data consistency
--  DevOps required

--------------------------------------------------------------------------------
--

>>  Microservices Characteristics

===========

1  Independently Deployable
--  Each service is deployed separately.

2  Loosely Coupled
--  A change in one service does not affect others.

3  Database per Service
--  Each service has its own database.

4  API Communication
--  Services communicate using REST, gRPC, or messaging.

5  Decentralized Governance
--  Each team can choose its own technology stack.

--------------------------------------------------------------------------------
--

>>  Benefits & Challenges

===========

>>  Benefits

--  Fast Development : Teams can work on different services simultaneously,
which speeds up development.

--  Easy Scaling : Individual services can be scaled independently based on
demand.

--  Fault Isolation : If one service fails, it does not crash the entire system.

--  Better Team Ownership : Each team owns and manages its service end-to-end.

===========
```

>> Challenges

-- Distributed Debugging : Debugging becomes harder because multiple services are involved.

-- Network Failure : Services communicate over the network so network issues can affect the system.

-- Data Consistency : Maintaining consistent data across multiple services is complex.

-- Security Complexity : Managing authentication, authorization, and secure communication across services is more difficult.

==========

>> Domain-Driven Design (DDD)

It helps in structuring code according to real business problems, making the system easier to understand, maintain, and scale. It is especially useful in complex microservices architectures where clear domain boundaries are needed.

Spring Cloud

>> Spring Cloud is an ecosystem of tools used to build and manage microservices easily. It provides ready-made solutions for common distributed system problems.

==========

>> Components

-- Config Server – Centralized configuration management for all services.

-- Netflix Eureka – Service registration and discovery.

-- Gateway – Single entry point for routing requests to services.

-- Netflix Ribbon – Client-side load balancing between service instances.

-- OpenFeign – Simplifies inter-service REST API calls.

-- Resilience4j – Handles circuit breaker, retry, and fault tolerance.

==========

>> Service Discovery (Eureka)

-- Managing service IP addresses manually is difficult because services scale up/down and their IPs or ports can change frequently. That’s why a Service Registry is used.

-- Netflix Eureka allows services to register themselves and discover other services automatically.

==========

>> Why It’s Important :

-- No need to manually manage IP addresses.

-- Supports dynamic scaling (new instances auto-register).
-- Improves system flexibility and reliability.

==========

>> Working :

-- When a service starts → it registers with Eureka.
-- Eureka stores all service instance details.
-- Other services query Eureka to find service locations.

==========

>> Architecture :

Eureka Server
   ↑
Service A | -- Eureka Server → Acts as a central registry.
Service B | -- Service A, B, C → Register their details with Eureka.
Service C |

>> How It Works :

-- When Service A starts, it registers itself with Eureka.
-- The same process is followed by Service B and Service C.
-- Eureka stores the IP address and port of all services.
-- If Service A needs to call Service B, it asks Eureka for B’s address.

==========

>> Eureka Server Code : pom.xml dependency

```
<dependency>
 <groupId>org.springframework.cloud</groupId>
 <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

==========

>> Load Balancing (Ribbon)

-- It is a client-side load balancer.
-- It distributes requests across multiple service instances.

>> Example

Suppose Order Service is running in 3 instances:
Order-1, Order-2, Order-3

When requests come, Ribbon does load balancing.

Round Robin Meaning

Request 1 → Order-1
Request 2 → Order-2
Request 3 → Order-3
Request 4 → Again Order-1

👉 It sends requests one by one to each instance in rotation, so load is equally distributed.

==========

>> Circuit Breaker (Resilience4j)

-- The Circuit Breaker is a failure-handling pattern used in microservices to stop continuous calls to a failing service.

-- Resilience4j helps prevent cascading failures, where one failed service affects the entire system.

>> Why It Is Needed

-- Imagine Service A calls Service B.

-- If Service B keeps failing and Service A keeps retrying, system resources (threads, memory, connections) get exhausted.

-- This can crash the whole system.

-- Circuit Breaker solves this by stopping calls temporarily when failures increase.

>> States of Circuit Breaker

-- Closed (Normal State)
-- Open (Blocking State)
-- Half-Open (Testing State)

--

>> Microservices Security Challenges

==========

>> Issues

-- Service-to-service authentication – Securing communication between internal services.

-- Data interception – Data can be intercepted while traveling over the network.

-- Token leakage – Access tokens may get exposed or misused.

-- API attacks – APIs can be targeted with attacks like injection, brute force, etc.

==========

>> Solutions

-- OAuth2 – Secure authorization framework.

-- JWT (JSON Web Token) – Token-based authentication mechanism.

-- API Gateway security – Centralized authentication, rate limiting, filtering.

-- HTTPS – Encrypts data in transit for secure communication.

--

>> Service-to-Service Authentication

-- In microservices, securing internal service calls is very important. Each

service must verify that the request coming from another service is trusted and authenticated.

==========

>> JWT Flow

User → Auth Service → JWT Token
Token → Other Services

-- User logs in → goes to Auth Service.
-- Auth Service generates a JWT token.
-- Token is returned to the user/client.
-- Client sends the same token while calling other services.
-- Each service validates the token before processing the request.

==========

>> Token Process

-- Token Generate → Created by Auth Service after login.

-- Token Validate → Verified by every microservice on each request.

==========

>> Request Architecture Flow

Client
  ↓
API Gateway   →   Entry point, forwards request.
  ↓
Auth Service (JWT)   →   Generates & validates JWT.
  ↓
Microservices   →   Process business logic.
  ↓
DB per service   →   Separate databases.
  ↓
Eureka Discovery   →   Service discovery.
  ↓
Resilience4j Protection   →   Failure protection.

--